

**AI-POWERED REAL-TIME POTHOLE DETECTION AND HAZARD
RISK ASSESSMENT SYSTEM FOR MICRO-MOBILITY (BICYCLE AND
ELECTRIC SCOOTER) USERS**

MUHAMMAD AL IEQBAL NUR HAQIEMI BIN ABDUL KHALID

UNIVERSITI TEKNIKAL MALAYSIA MELAKA

BORANG PENGESAHAN STATUS LAPORAN

JUDUL: [AI-POWERED REAL-TIME POTHOLE DETECTION AND HAZARD RISK ASSESSMENT SYSTEM FOR MICRO-MOBILITY (BICYCLE AND ELECTRIC SCOOTER) USERS]

SESI PENGAJIAN: [2026 / 2027]

Saya: Muhammad Al Ieqbal Nur Haqiemi bin Abdul Khalid

mengaku membenarkan tesis Projek Sarjana Muda ini disimpan di Perpustakaan Universiti Teknikal Malaysia Melaka dengan syarat-syarat kegunaan seperti berikut:

1. Tesis dan projek adalah hakmilik Universiti Teknikal Malaysia Melaka.
2. Perpustakaan Fakulti Kecerdasan Buatan dan Keselamatan Siber dibenarkan membuat salinan untuk tujuan pengajian sahaja.
3. Perpustakaan Fakulti Kecerdasan Buatan dan Keselamatan Siber dibenarkan membuat salinan tesis ini sebagai bahan pertukaran antara institusi pengajian tinggi.
4. * Sila tandakan (✓)

☒

SULIT

(Mengandungi maklumat yang berdarjah keselamatan atau kepentingan Malaysia seperti yang termaktub di dalam AKTA RAHSIA RASMI 1972)

☒

TERHAD

(Mengandungi maklumat TERHAD yang telah ditentukan oleh organisasi / badan di mana penyelidikan dijalankan)

☒

TIDAK TERHAD

(TANDATANGAN PELAJAR)

(TANDATANGAN PENYELIA)

Alamat tetap: _____

Nama Penyelia

Tarikh: _____

Tarikh: _____

CATATAN: * Jika tesis ini SULIT atau TERHAD, sila lampirkan surat daripada pihak

[AI-POWERED REAL-TIME POTHOLE DETECTION AND HAZARD RISK
ASSESSMENT SYSTEM FOR MICRO-MOBILITY (BICYCLE AND E-
SCOOTER) USERS]

[MUHAMMAD AL IEQBAL NUR HAQIEMI BIN ABDUL KHALID]

This report is submitted in partial fulfillment of the requirements for the
Bachelor of [Computer Science (Artificial Intelligence)] with Honours.

FACULTY OF ARTIFICIAL INTELLIGENCE AND CYBER SECURITY
UNIVERSITI TEKNIKAL MALAYSIA MELAKA

[2026]

DECLARATION

I hereby declare that this project report entitled
**[AI-POWERED REAL-TIME POTHOLE DETECTION AND HAZARD RISK
ASSESSMENT SYSTEM FOR MICRO-MOBILITY (BICYCLE AND ELECTRIC
SCOOTER) USERS]**

is written by me and is my own effort and that no part has been plagiarized
without citations.

STUDENT : MUHAMMAD AL IEQBAL NUR
HAQIEMI BIN ABDUL KHALID

Date : _____

I hereby declare that I have read this project report and found
this project report is sufficient in term of the scope and quality for the award of
Bachelor of [Computer Science (Artificial Intelligence)] with Honours.

SUPERVISOR : TS. DR. WAN MOHD YA'AKOB WAN BEJURI

Date : _____

DEDICATION

This final year project report is dedicated to my beloved parents, whose unconditional love, endless sacrifices, prayers, and constant support have been the cornerstone of my life and the driving force behind my academic achievements.

It is also dedicated to my esteemed supervisor and lecturers, whose patience, invaluable guidance, and intellectual mentorship have shaped my understanding and inspired me to strive for academic and professional excellence.

Finally, this work is dedicated to my fellow peers and friends, who shared the challenges and triumphs of this journey, offering constant encouragement, support, and companionship that made these years truly memorable and meaningful.

ACKNOWLEDGEMENTS

First and foremost, I express my deepest gratitude to Almighty God for granting me the strength, health, wisdom and patience to complete this Final Year Project and compile this thesis successfully.

I would like to express my sincere appreciation and profound gratitude to my supervisor Ts. Dr. Wan Mohd Ya'akob Wan Bejuri, for his invaluable guidance, academic supervision, patience and constant encouragement throughout the duration of this project. His constructive feedback, insightful advice and expert knowledge in artificial intelligence and software engineering were critical to overcoming research challenges and successfully deploying the edge deep learning models and mobile application.

I am also highly indebted to the Faculty of Artificial Intelligence and Cyber Security (FAIX) and Universiti Teknikal Malaysia Melaka (UTeM) for providing the academic platform, laboratory facilities and resources that made this project possible.

Finally, my heartfelt thanks go to my beloved parents for their unconditional love, moral support, financial assistance and constant prayers that motivated me throughout my academic journey. I also extend my gratitude to my classmates and fellow peers for their direct assistance, technical suggestions and shared encouragement, all of which contributed significantly to the successful completion of this final year project.

ABSTRACT

Micro-mobility solutions, particularly bicycles and electric stand-up scooters have become integral components of modern urban transit systems. While these transport modes offer significant environmental benefits and address first and last mile logistics, their safety is routinely compromised by decaying road infrastructure. Potholes and other surface anomalies pose severe risks to micro-mobility riders due to the vehicle's small wheel diameters and rigid suspension geometries. Standard vehicular safety technologies are designed for passenger cars and do not cater to these vulnerable road users, while existing mapping applications lack the real-time, localized hazard warnings necessary to prevent accidents. Furthermore, accelerometer-based detection methods are reactive, registering road anomalies only after an impact has occurred. To address these safety gaps, this paper proposes a proactive, vision-based road hazard detection and risk assessment system designed to run locally on handlebar-mounted mobile devices. I have utilized a lightweight YOLOv8 Nano object detection model optimized for edge execution via TensorFlow Lite with INT8 quantization, operating fully offline to ensure zero latency and protect user privacy. To mitigate alert fatigue, we formulate a dynamic, physics-inspired Risk Assessment Algorithm that calculates a Risk Index by correlating the detected pothole's dimensions with the vehicle's instantaneous GPS speed and wheel diameter. This paper details the system's architecture, including a concurrent execution model using background isolates to prevent UI lag. Our implementation achieves real-time inference at over 15 Frames Per Second on standard mobile processors with a pipeline latency under 200 ms, successfully warning riders of high-risk hazards through high-contrast visual overlays and audio alerts.

TABLE OF CONTENTS

	PAGE
DECLARATION.....	II
DEDICATION.....	III
ACKNOWLEDGEMENTS.....	IV
ABSTRACT	V
TABLE OF CONTENTS.....	VI
LIST OF TABLES	IX
LIST OF FIGURES	X
LIST OF ABBREVIATIONS	XI
LIST OF ATTACHMENTS.....	12
CHAPTER 1: INTRODUCTION.....	13
1.1 Introduction.....	Error! Bookmark not defined.3
1.2 Problem Statement	Error! Bookmark not defined.4
1.3 Objectives of the Project.....	Error! Bookmark not defined.4
1.4 Scope of the Project	Error! Bookmark not defined.5
1.5 Project Significance	Error! Bookmark not defined.5
1.6 Expected Output.....	Error! Bookmark not defined.5
1.7 Report Organisation	Error! Bookmark not defined.6
1.8 Summary	Error! Bookmark not defined.6

CHAPTER 2: LITERATURE REVIEW AND PROJECT METHODOLOGY 17

2.1	Introduction.....	17
2.2	Facts and Findings	17
2.2.1	Domain.....	17
2.2.2	Existing System	18
2.2.3	Technique.....	19
2.3	Project Methodology.....	20
2.4	Project Requirements	21
2.4.1	Software Requirement	21
2.4.2	Hardware Requirement	21
2.5	Project Schedule and Milestones	22
2.6	Summary	22

CHAPTER 3: REQUIREMENT ANALYSIS..... 23

3.1	Introduction.....	23
3.2	Problem Analysis	2Error! Bookmark not defined.
3.3	Requirement Analysis	27
3.3.1	Data Requirement	27
3.3.2	Functional Requirement.....	30
3.3.3	Non-Functional Requirement.....	31
3.3.4	Others Requirement	32
3.4	Summary	32

CHAPTER 4: DESIGN	33
4.1 Introduction.....	33
4.2 High-Level Design.....	33
4.2.1 System Architecture.....	33
4.2.2 User Interface Design	36
4.3 AI Component Design	41
4.3.1 YOLOv8 Model Configuration.....	41
4.3.2 TFLite Execution Flow	41
4.4 Software Design.....	42
4.5 Summary	42
 CHAPTER 5: RESULTS AND DISCUSSION	 43
5.1 Introduction.....	43
5.2 Evaluation of AI Techniques	43
5.3 Testing of Functional Requirement	50
5.4 Summary	51
 CHAPTER 6: CONCLUSION AND RECOMMENDATIONS	 52
6.1 Observation on Weaknesses and Strengths.....	52
6.2 Propositions for Improvement	53
6.3 Project Contribution.....	54
6.4 Summary	54
 REFERENCES.....	 55
 APPENDICES	 58

LIST OF TABLES

	PAGE
Table 2.1: Comparison of Existing Hazard Detection Systems	18
Table 2.2: Comparison of Object Detection Architectures	19
Table 3.1: Ride History Data Schema (rides.json)	28
Table 3.2: Application Settings Schema (settings.json)	29
Table 5.1: Comparative Performance of YOLOv8 Model Scales.....	44
Table 5.2: Functional Test Case Specifications	50

LIST OF FIGURES

	PAGE
Figure 3.1: System Workflow Flowchart (Section 3.2)	26
Figure 3.2: Functional Use Case Diagram (Section 3.3.2)	31
Figure 4.1: Compact Three-Tier System Architecture (Section 4.2.1).....	3Error!
Bookmark not defined.	
Figure 4.2: System Sequence Diagram (Section 4.2.1).....	35
Figure 4.3: App Navigation Flow Diagram (Section 4.2.2)	36
Figure 4.4: Rider Dashboard Interface	38
Figure 4.5: Pre-Recording Landing Screen	39
Figure 4.6: Active Scanning HUD with Detections	39
Figure 4.7: Aggregate Hazard Map.....	40
Figure 4.8: Ride Details and Polyline Summary	40
Figure 5.1: Bounding Box and Classification Loss Curves (results.png).....	46
Figure 5.2: YOLOv8 Object Detection Confusion Matrix (confusion_matrix.png)	47
Figure 5.3: Precision-Recall (PR) Curve (BoxPR_curve.png)	48
Figure 5.4: Dataset Annotation Labels and Spatial Distribution (labels.jpg)	49

LIST OF ABBREVIATIONS

AI	-	Artificial Intelligence
API	-	Application Programming Interface
APK	-	Android Package Kit
CNN	-	Convolutional Neural Network
CUDA	-	Compute Unified Device Architecture
FAIX	-	Faculty of Artificial Intelligence and Cyber Security
FPS	-	Frames Per Second
GPS	-	Global Positioning System
HUD	-	Head-Up Display
IEEE	-	Institute of Electrical and Electronics Engineers
INT8	-	8-bit Integer Quantization
IoU	-	Intersection over Union
JSON	-	JavaScript Object Notation
mAP	-	mean Average Precision
OSM	-	OpenStreetMap
PSM	-	Projek Sarjana Muda (Final Year Project)
RAM	-	Random Access Memory
RI	-	Risk Index
TFLite	-	TensorFlow Lite
UTeM	-	Universiti Teknikal Malaysia Melaka
YOLO	-	You Only Look Once

LIST OF ATTACHMENTS

Attachment A	Mobile Application Source Code Repository
Attachment B	Analysis of data collection
Attachment C	Pothole Object Detection Training Dataset

CHAPTER 1: INTRODUCTION

1.1 Introduction

The rapid expansion of urban centers has triggered a corresponding rise in micro-mobility solutions, notably electric scooters and bicycles, which offer clean, efficient, and flexible alternatives to traditional vehicular transit. However, urban road infrastructure has failed to evolve alongside these micro-mobility networks. Surface defects, such as potholes, present a severe hazard to light, two-wheeled vehicles. Unlike passenger automobiles, which feature large pneumatic tires and complex suspension systems capable of absorbing significant impacts, micro-mobility vehicles generally have small wheel diameters and rigid frames. A minor asphalt depression that causes a negligible bounce in a car can destabilize a scooter or bicycle, leading to catastrophic accidents, severe rider injuries, or structural frame failures.

To address this issue, modern safety technology must shift from reactive monitoring to proactive prevention. Traditional GPS applications provide route guidance but fail to offer granular, real-time hazard warnings. Similarly, standard dashboard cameras only document accidents post-occurrence. A real-time, vision-based hazard detection system mounted directly on the vehicle handlebars is required to identify road defects before a collision occurs and assess threat severity based on the rider's immediate velocity and vehicle constraints.

1.2 Problem Statement

The safety framework for micro-mobility users suffers from three critical vulnerabilities. First, the mechanical design of bicycles and electric scooters specifically their small wheels and lack of substantial suspension renders riders highly susceptible to minor road defects, where hitting a standard-sized pothole often results in a loss of control or a front-wheel lockup. Second, contemporary navigation platforms, such as Google Maps or OpenStreetMap, provide static route suggestions but lack real-time, crowdsourced, or localized surface quality assessments, leaving riders unaware of immediate road degradation. Third, current vehicle safety systems are primarily reactive, which record impact data or register GPS coordinates after an accident has occurred, failing to deliver immediate, contextual warnings that allow the rider to execute safe evasive maneuvers before reaching the hazard.

1.3 Objectives of the Project

To resolve these challenges, the project has established three primary research objectives. The first objective is to study, train and optimize a lightweight convolutional neural network utilizing the YOLOv8 architecture to achieve real-time, vision-based pothole detection on resource-constrained mobile hardware. The second objective is to design and develop a fully offline Android application in Flutter that integrates camera feed processing, real-time GPS speed measurements and an exponential speed-risk calculation algorithm to deliver visual and audio alerts to the rider. The third objective is to systematically evaluate the trade-offs between three different model scales which are YOLOv8 Nano, YOLOv8 Small and YOLOv8 Medium and will be quantified through training latency, Mean Average Precision, post-quantization model size and on-device inference speed.

1.4 Scope of the Project

The scope of this project is strictly defined across data, hardware and software parameters. Geographically and operationally, the system is designed for handlebar-mounted Android smartphones deployed on bicycles or electric scooters, processing camera feeds in a forward-facing orientation. The data scope is restricted to the Pothole-vhmow dataset sourced from Roboflow Universe, compiled and pre-processed to target pothole defects under variable daylight, shadowing, and viewing angles. Software execution is constrained to a fully offline environment, executing inference locally on the device CPU using 8-bit quantized TensorFlow Lite (TFLite) models, which ensures zero latency and constant protection of user location privacy without requiring cellular connectivity.

1.5 Project Significant

This research provides dual-fold contributions to individual safety and urban planning. For micro-mobility riders, the proactive warning system directly mitigates the risk of accidents by providing a visual HUD overlay and high-frequency audio beeps when approaching a pothole at unsafe speeds. For municipalities and the broader public, the application archives ride telemetry and detected hazard coordinates locally, establishing a foundation for aggregate local mapping that can inform road maintenance crews of critical defects, improving overall urban infrastructure safety.

1.6 Expected Output

The project outputs consist of two core components. The first is a set of optimized, INT8-quantized YOLOv8 object detection models exported to the TFLite format. The second is a fully functional Android mobile application built using the Flutter framework. This application features a real-time recording HUD, a dynamic sound synthesis module for contextual audio warnings and an offline history map interface utilizing OpenStreetMap tiles rendered via a custom dark-theme filter matrix to plot historical commute coordinates and hazard pins.

1.7 Report Organisation

This report is divided into six chapters. Chapter 1 introduces the project background, objectives, scope, and significance. Chapter 2 details the literature review, analysing existing road damage detection systems and presents the project methodology. Chapter 3 outlines the requirement analysis, specifying the functional, non-functional, and data demands. Chapter 4 describes the high-level and low-level architectural design of the application. Chapter 5 compiles the experimental results, comparing the trained YOLOv8 Nano, Small, and Medium models and validating the dynamic risk index. Finally, Chapter 6 summarizes the research findings and proposes recommendations for future work.

1.8 Summary

In summary, this chapter established the foundation of the research by highlighting the physical vulnerabilities of micro-mobility users to road defects and outlining the proposed vision-based solution. The objectives, scope, and expected outputs were defined to focus the development on a real-time, offline Android application. The next chapter will provide an analysis of existing literature and detail the methodology used to train the models and implement the software.

CHAPTER 2: LITERATURE REVIEW AND PROJECT METHODOLOGY

2.1 Introduction

Developing a real-time, on-device road hazard detection system requires a synthesis of computer vision, mobile software engineering, and spatial data modelling. This chapter reviews the physical challenges within the micro-mobility domain, evaluates existing detection systems, compares standard deep learning models, and details the project methodology and technical requirements.

2.2 Facts and Findings

2.2.1 Domain

Micro-mobility vehicles, comprising bicycles and electric scooters are physically distinct from passenger vehicles. Their key constraint is the wheel-to-hazard diameter ratio. A standard electric scooter utilizes tires between 8.5 and 10 inches in diameter, whereas a standard passenger car tire exceeds 26 inches. Consequently, a pothole with a depth of 2 inches presents a minor nuisance to a car but represents a major obstacle to a scooter, equal to roughly 20% of its tire diameter. Additionally, these lightweight vehicles lack passive damping systems, transferring the kinetic energy of an impact directly to the handlebars and the rider. This mechanical vulnerability necessitates proactive detection, allowing the rider to circumvent the defect rather than relying on impact mitigation.

2.2.2 Existing System

Existing road hazard mapping systems fall into three categories: tactile/vibration-based, active range-finding, and visual inspection. Tactile systems utilize smartphone accelerometers to log anomalies. While computationally cheap, they are inherently reactive, registering the hazard only after the vehicle has experienced the impact. Active range-finding systems using LiDAR provide high accuracy but are cost-prohibitive and consume substantial power, making them unsuitable for consumer mobile deployment. Visual inspection systems are typically deployed on municipal vehicles using high-definition cameras and cloud processing, which lacks real-time warning capabilities for individual riders.

Parameters	Accelerometer System	LiDAR Systems	Municipal Survey	Proposed App
Detection Type	Reactive (post-impact)	Proactive (pre-impact)	Proactive	Proactive (pre-impact)
Cost	Low (Built-in sensor)	Very High (LiDAR unit)	High (Specialized vehicle)	Low (Standard smartphone)
Edge Processing	Offline	Offline/ Local	Offline/ Upload	Offline (Local TFLite)
Warning Latency	N/A (No warning)	Medium	N/A (Offline report)	Very Low (<200ms)
Network Dependency	Low	Low	High	None (Fully Offline)

Table 2.1: Comparison of Existing Hazard Detection Systems

2.2.3 Technique

Real-time visual object detection on mobile hardware requires models with low parameter counts fast inference speeds and high representation capacity. Traditional two-stage detectors, such as Faster R-CNN, achieve high accuracy but have excessive inference latencies that prevent real-time mobile deployment. One-stage detectors, such as SSD MobileNet and the YOLO series, consolidate bounding box regression and classification into a single pass. YOLOv8 optimizes this approach by employing an anchor-free detection head and a decoupled structure, separating classification and regression branches to maximize mean Average Precision (mAP) while minimizing computational complexity.

Architecture	Parameter Count	GPU Latency	mAP50	Edge Suitability
Faster R-CNN	~40M	High	~82%	Poor (Too heavy)
SSD MobileNetV2	~3.4M	Very Low	~70%	High (Low accuracy)
YOLOv8 Nano	~3.2M	Very Low	~89%	Excellent
YOLOv8 Small	~11.2M	Low	~92%	High
YOLOv8 Medium	~25.9M	Medium	~94%	Medium

Table 2.2: Comparison of Object Detection Architectures

2.3 Project Methodology

This project adopts a structured methodology based on the CRISP-DM (Cross-Industry Standard Process for Data Mining) framework, adapted for deep learning deployment. The methodology is divided into three distinct phases.

Phase 1: Data Acquisition and Pre-processing. The Pothole-vhmow dataset (v18), consisting of 1,302 pothole images, was sourced from Roboflow Universe. Using the Roboflow API, the dataset was pre-processed to target pothole defects. Pre-processing steps included auto-orientation of pixel data, EXIF-orientation stripping, and resizing to 640x640 pixels. The final dataset was split into training, validation, and testing partitions.

Phase 2: Model Training and Quantization. Three scales of YOLOv8 (Nano, Small, and Medium) were trained for 50 epochs with an input size of 320x320 pixels using PyTorch on a local workstation. Following training, the models were converted to the TensorFlow SavedModel format and subjected to INT8 Post-Training Quantization (PTQ). This quantization maps the 32-bit floating-point weights and activations to 8-bit integers using a representative dataset of 100 images to calibrate dynamic ranges.

Phase 3: Android Integration. The quantized `.tflite` models were integrated into the Flutter application. Using the TFLite Interpreter API, the app passes camera frames directly to the model, parses the output bounding boxes and calculates coordinates relative to the screen size. GPS telemetry is captured via the Geolocator package, and both telemetry and detections are processed by the alert logic.

2.4 Project Requirements

2.4.1 Software Requirements

The development stack comprises:

- Flutter SDK (Dart) for cross-platform UI construction and hardware bindings.
- PyTorch and the Ultralytics API for training YOLOv8 architectures.
- TensorFlow Lite (TFLite) library for mobile compilation and quantization.
- Android SDK (API Level 21+) for compiling the final application.
- VS Code and Android Studio as the integrated development environments.

2.4.2 Hardware Requirements

The hardware environment includes:

- Workstation equipped with an NVIDIA GeForce RTX 3050 Laptop GPU (4GB VRAM) for CUDA-accelerated model training.
- Android smartphone (test device: Xiaomi 22101316UG, 8GB RAM, octa-core CPU) for deploying and validating the mobile application.

2.5 Project Schedule and Milestones

The project is executed over two academic semesters. The first semester focuses on proposal validation, data collection, literature synthesis and preliminary model testing. The second semester focuses on workstation-based GPU model training, edge quantization, Flutter application development, local storage persistence implementation, field testing, and report compilation.

2.6 Summary

This chapter analysed the mechanical constraints of micro-mobility, justified the use of YOLOv8 and TFLite through comparative analysis, and detailed the CRISP-DM methodology. The hardware and software requirements were defined to establish a clear development boundary. The next chapter will detail the requirement analysis, mapping the operational requirements of the system.

CHAPTER 3: PROJECT METHODOLOGY

3.1 Introduction

Before designing the application components, the system's requirements must be modelled to align with physical safety constraints and the smartphone's hardware limits. This chapter provides a problem analysis, defines the mathematical alert logic, and details the data, functional, and non-functional requirements.

3.2 Problem Analysis

Under normal riding conditions, a cyclist or scooter rider relies on manual visual scanning. This process becomes unreliable under high speeds, direct sunlight, or fatigue. If a rider fails to notice a pothole, the collision occurs without warning, transferring impact energy to the vehicle frame. The proposed system acts as an active safety layer. It continuously captures video frames and GPS speed, performs on-device object detection, and runs a Risk Assessment Algorithm. If the computed Risk Index exceeds a critical threshold, the system delivers immediate visual and audible feedback, enabling the rider to adjust their course.

The threat severity of a pothole is mathematically modeled by the Risk Index (RI) formula:

$$RI = \left(\frac{S_{detected}}{S_{threshold}} \right) \times \left(\frac{V_{current}}{V_{limit}} \right)^2 \times 100\%$$

Where the parameters are defined as:

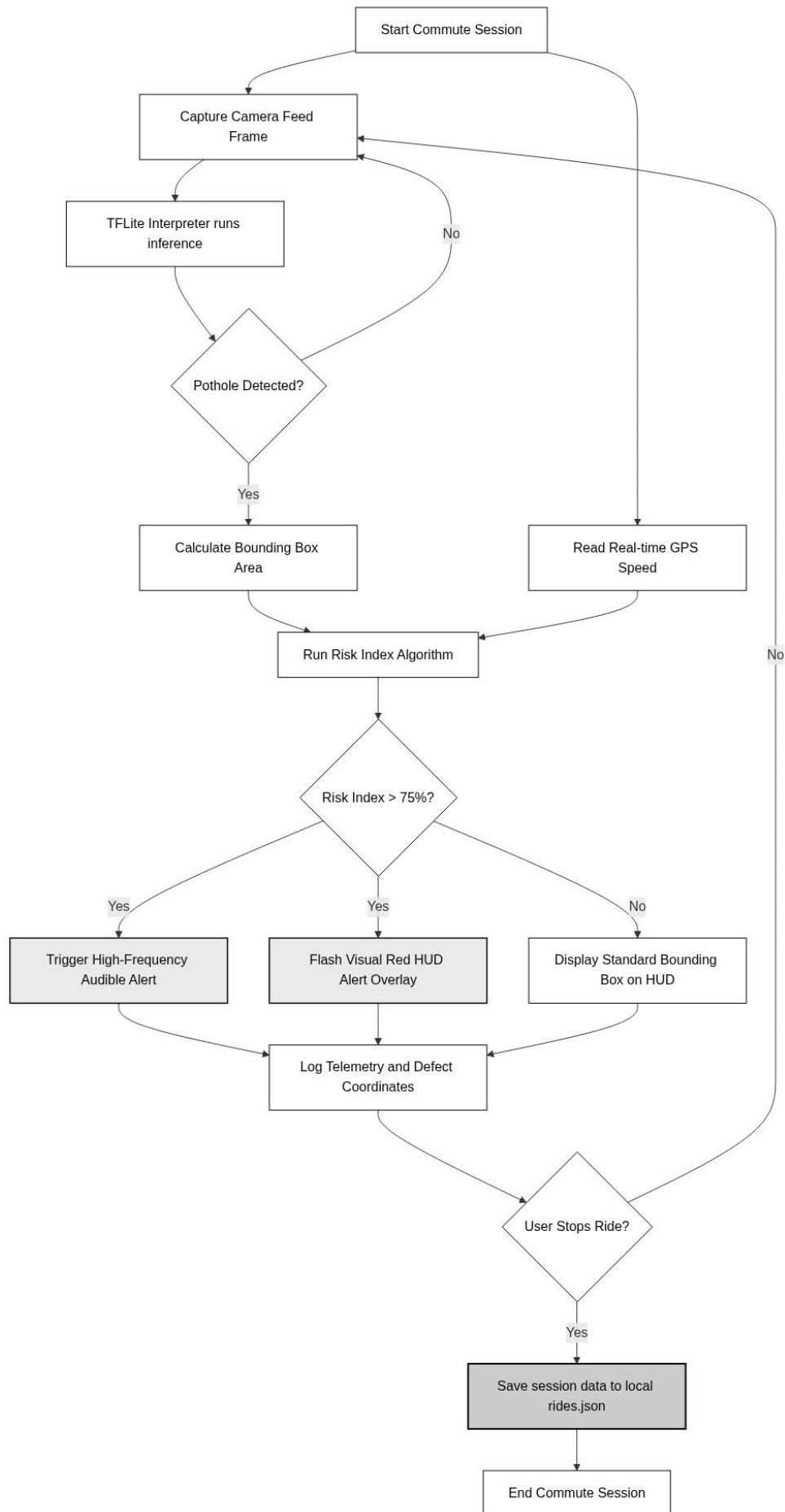
- $S_{detected}$: The surface area of the bounding box generated by the YOLOv8 model in pixels.
- $S_{threshold}$: The wheel size calibration constant. If E-scooter is selected, a smaller constant is used. (e.g. 8.5 Inches), which increases the baseline risk score. If Bicycle is selected (26 inches), a larger constant is used, which dampens the risk.

- $V_{current}$: The real-time velocity retrieved from the smartphone's GPS receiver in km/h.
- V_{limit} : The speed limit constant (default: 25 km/h)

By squaring the velocity ratio, the formula mirrors the kinetic energy equation ($K.E. = \frac{1}{2}mv^2$). This design ensures that the calculated risk scales exponentially with speed, recognizing that hitting a pothole at 20 km/h is four times more destructive than hitting the same pothole at 10 km/h.

System Workflow Flowchart

This flowchart represents the real-time processing loop during a commute session:



3.3 Requirement Analysis

3.3.1 Data Requirements

The system requires three primary data inputs: camera frame buffers (converted to RGB image arrays), GPS latitude/longitude coordinates, and speed values (m/s converted to km/h).

For persistence, the application utilizes two local JSON files:

1. **rides.json**: Stores historical ride records. The schema includes a unique ride ID, date/time stamp, total distance, duration, average speed, a list of polyline coordinate pairs representing the route, and a list of logged pothole markers containing latitude, longitude, and computed Risk Index.
2. **settings.json**: Stores user configurations, including vehicle type selection (E-Scooter or Bicycle), alert volume level, speed simulation toggle, and simulated target speed.

Local Data Schemas Reference Tables

These tables show the exact data fields managed by the database layer:

Field Name	Data Type	Description
id	String	Unique UUID for the commute session.
title	String	Time-of-day formatted name (e.g., "Morning Ride ").
date	String	ISO-8601 formatted timestamp of the ride.
distance	Float	Total ride distance completed (in km).
duration	Integer	Total commute length (in seconds).
avgSpeed	Float	Calculated average velocity (in km/h).
maxSpeed	Float	Highest recorded velocity (in km/h).
polyline	Array of lists	Array of [latitude, longitude] tracking the route path.
potholes	Array of objects	List of hazards logged, containing coordinates and Risk Index.

Table 3.1: Ride History Data Schema (rides.json)

Field Name	Data Type	Description
vehicleType	String	Toggle value: "E-Scooter" or "Bicycle".
alertVolume	Float	Range of 0.0 (mute) to 1.0 (maximum beep volume).
simulateSpeed	Boolean	True enables GPS speed override simulation for testing.
targetSpeed	Float	Target cruise speed (in km/h) for the simulation slider.

Table 3.2: Application Settings Schema (setting.json)

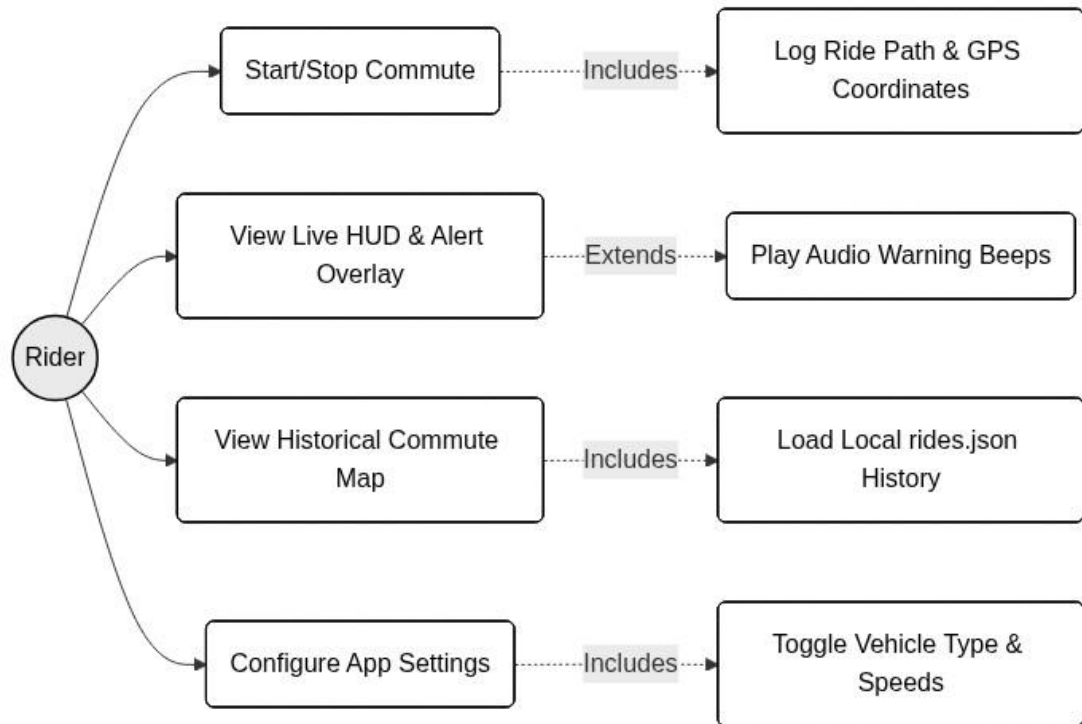
3.3.2 Functional Requirements

The application must perform the following core functions:

- **Camera Stream Capture:** Retrieve frames from the device camera and stream them to the TFLite Interpreter.
- **Local Inference:** Run the TFLite model locally to detect potholes, producing class labels, confidence scores, and bounding boxes.
- **GPS Telemetry Processing:** Monitor the device GPS, calculate vehicle speed, and append coordinates to the active route polyline.
- **Dynamic Alerting:** Compute the Risk Index using the active speed and model detections, triggering a visual red screen flash and synthetic audio beeps if the risk exceeds 75%.
- **Session Persistence:** Record the commute path and logged hazard locations to local storage when the user taps "Save Ride."
- **Route Rendering:** Load historical routes and render them on a dynamic map with color-coded hazard markers (green for low risk, orange for medium, red for high).

Functional Use Case Diagram

This diagram maps rider actions to functional subsystems:



3.3.3 Non-Functional Requirements

The system must satisfy several performance and operational constraints:

- **Latency:** The combined latency of image capture, model inference and risk computation must remain below 200 milliseconds to provide a warning before the vehicle reaches the defect.
- **Offline Capability:** The core detection, alert and persistence logic must function entirely without internet connectivity, relying on local resources.
- **Sunlight Readability:** The user interface must employ a high-contrast dark theme to ensure HUD metrics and map lines remain legible under direct sunlight.
- **Resource Optimization:** The application must release camera and GPS resources immediately upon stopping a recording to conserve device battery.

3.3.4 Others Requirement

The selection of Flutter and TFLite is justified by cross-platform compatibility and edge performance. Flutter allows native compilation on Android with access to low-level hardware APIs (Camera, Location) without intermediate bridges. TensorFlow Lite provides specialized hardware acceleration via the Android Neural Networks API (NNAPI), enabling the model to utilize the device NPU or GPU for faster inference.

3.4 Summary

This chapter analyzed the requirements of the pothole detection application. The dynamic Risk Index formula was defined to scale threat levels with vehicle velocity, and the data, functional, and non-functional requirements were established. The next chapter will present the system's structural and interface designs.

CHAPTER 4: PROPOSED METHOD

4.1 Introduction

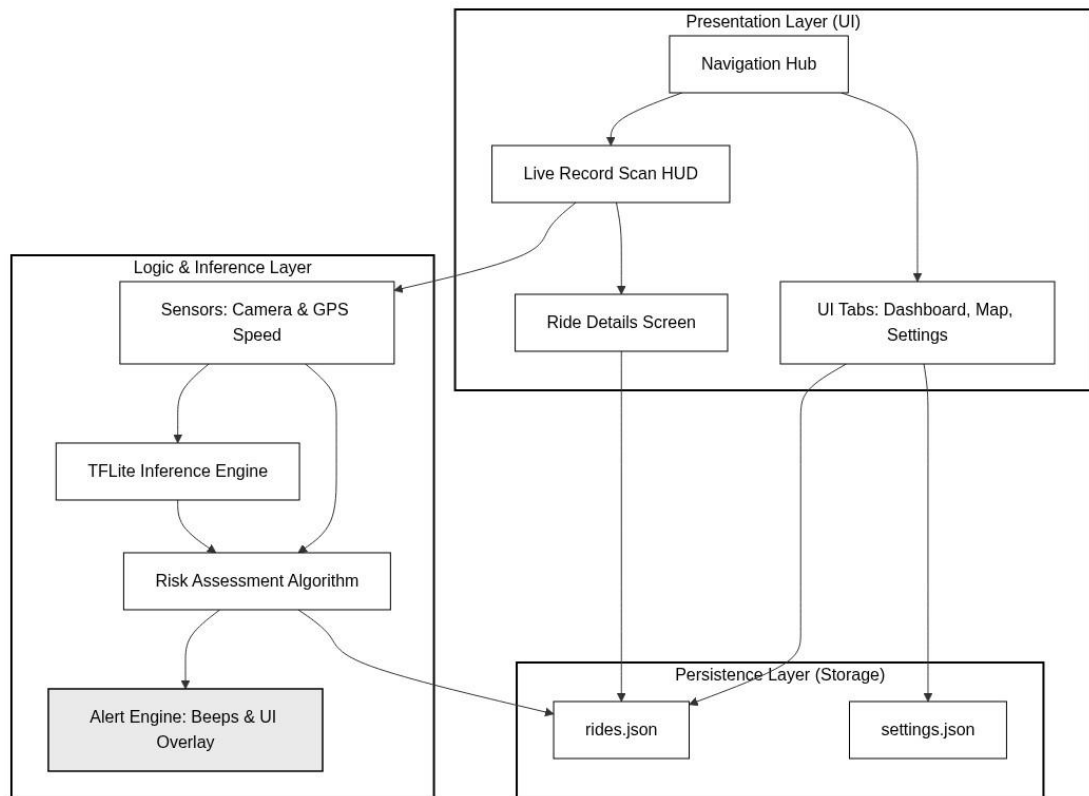
This chapter presents the architectural and component designs of the AI-powered real-time pothole detection and hazard risk assessment system. It defines the structural layout of the application, starting from the high-level system architecture down to the detailed low-level designs. The chapter covers the three-tier system architecture, user interface design (including input, output, and navigation flows) non-database data serialization schemas, and the deep learning component design (focusing on the YOLOv8 network configuration and the mobile TFLite execution logic).

4.2 High-Level Design

4.2.1 System Architecture

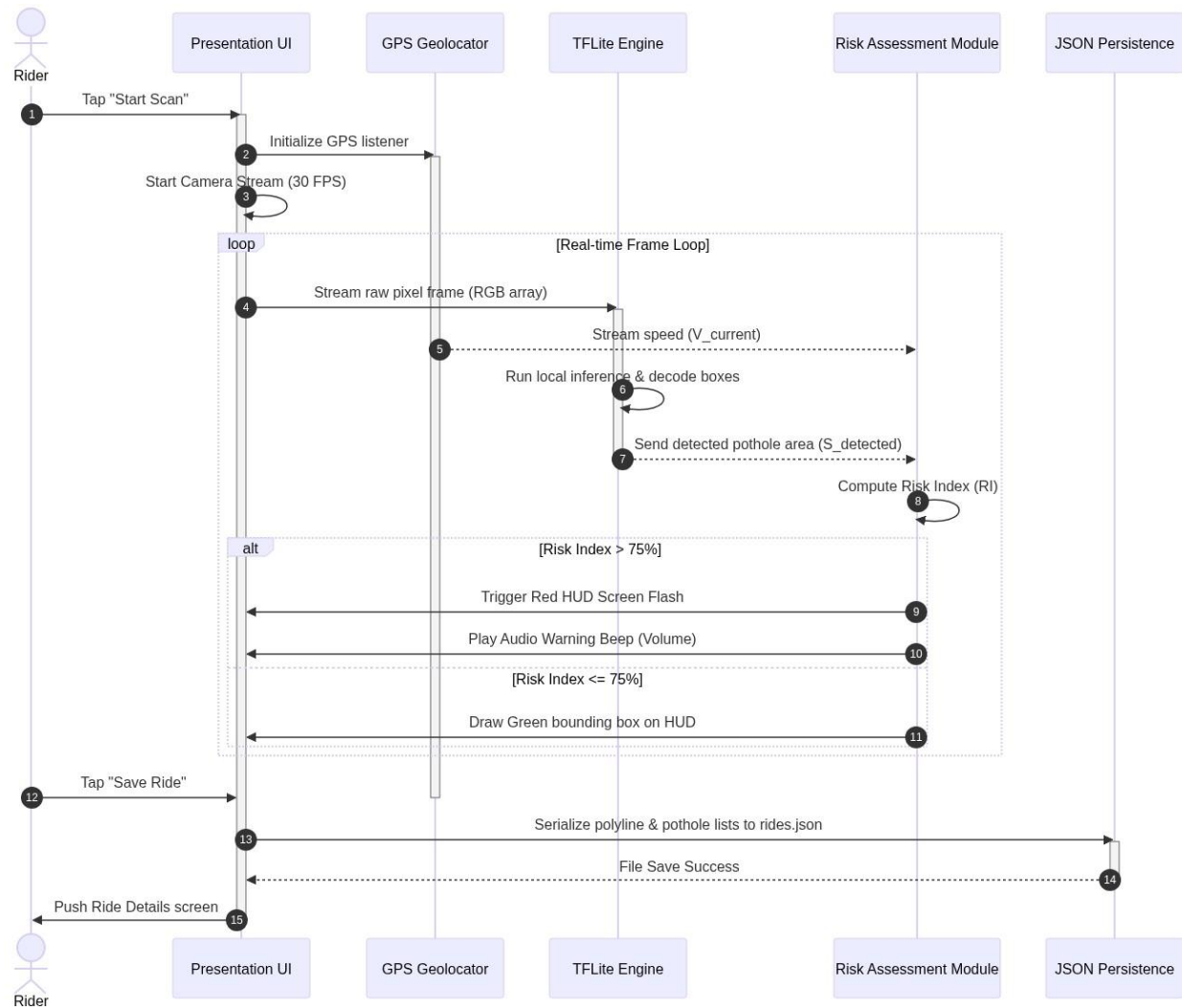
The application is structured using a three-tier architectural pattern, which isolates core visual elements from processing logic and data persistence. This modularity ensures that camera frames and GPS signals are processed asynchronously without blocking the user interface thread.

The diagram below visualizes the architectural layout:



System Sequence Diagram

This diagram shows the sequence of operations between subsystems when a riding commute is active:

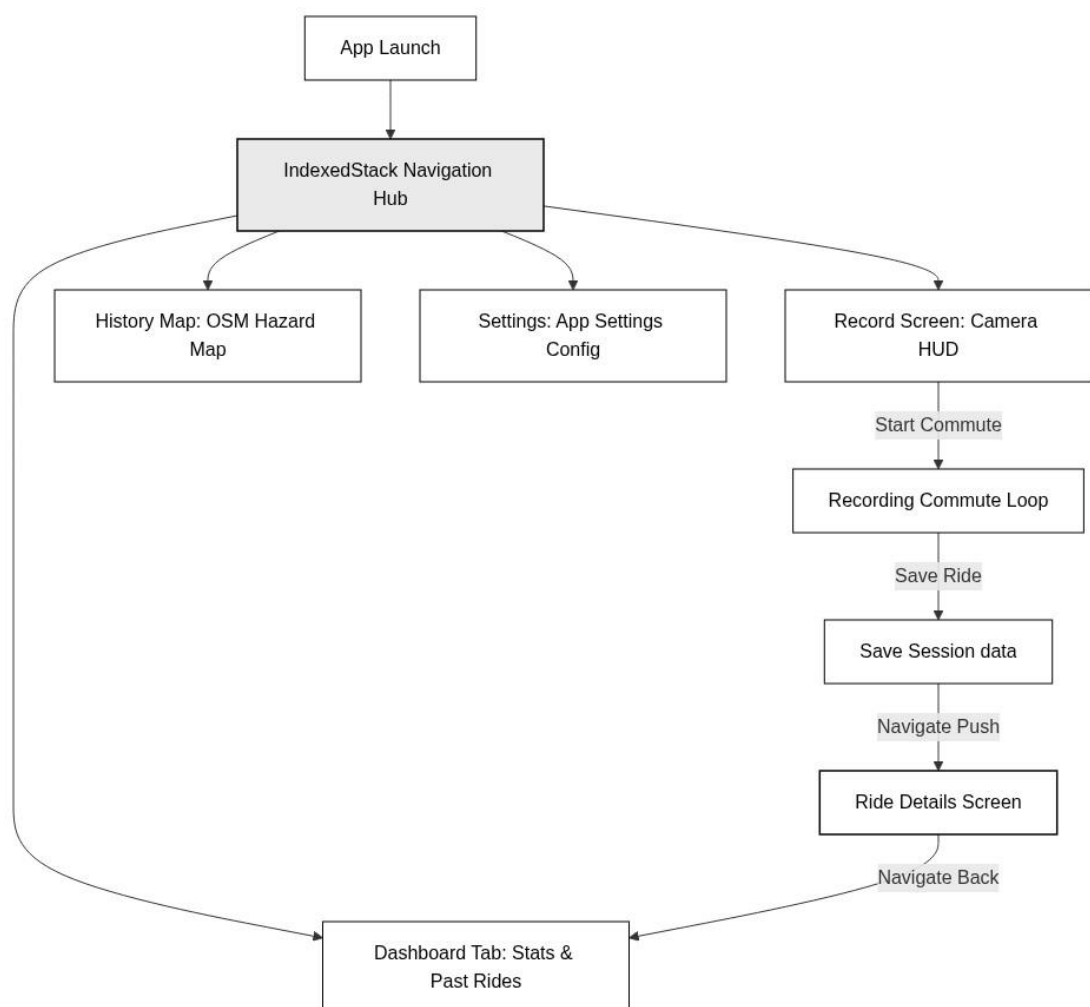


4.2.2 User Interface Design

The user interface is designed as a dark-theme dashboard structured around four core navigation tabs, optimized for high visibility under direct sunlight on handlebar mounts.

1. Navigation Flow Design

The diagram below maps the navigation path of the user through the application:



2. Input Design

System inputs are divided into hardware sensor feeds and user configurations.

- Sensor inputs include the raw 30 FPS camera frames and GPS coordinate streams.
- User inputs include the vehicle type selector (toggling between E-Scooter and Bicycle settings), the alert volume slider (0.0 to 1.0), the speed simulation toggle, and the target speed simulation slider (10 to 40 km/h).
- Validation rules require camera and location permissions to be granted before starting a ride. If permission is denied, the start button is disabled, and an error prompt is displayed. The simulated speed input is clamped to a maximum of 40 km/h to prevent out-of-bounds calculations in the risk algorithm.

3. Output Design

System outputs are designed to be immediate and non-intrusive.

- **Visual Outputs:** A bounding box drawn around the pothole with a label (e.g., "Pothole: 88%") is rendered on the HUD. If the Risk Index exceeds 75%, a semi-transparent red overlay flashes across the screen. On the map views, route paths are plotted as a blue polyline, and hazard locations are displayed as color-coded pins.
- **Audio Outputs:** Synthetic warnings play at variable frequencies; the beep frequency scales with the risk index, alerting the rider to slow down as their velocity increases.
- **Tabular Statistics:** Total distance (km), ride duration (seconds), average speed (km/h), and logged pothole counts are rendered in tables on the Dashboard feed and the Details screen.

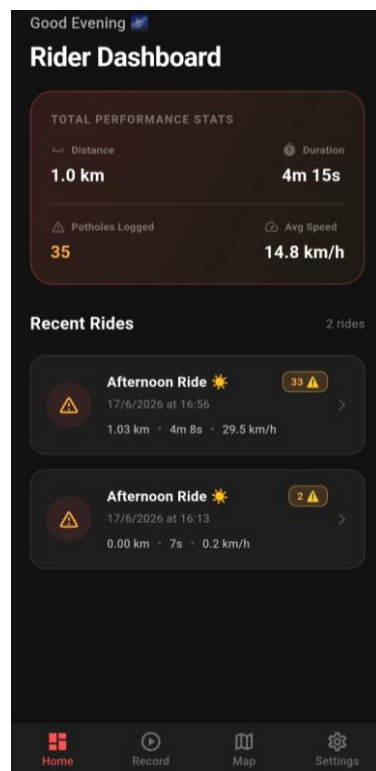


Figure 4.3: Rider Dashboard Interface

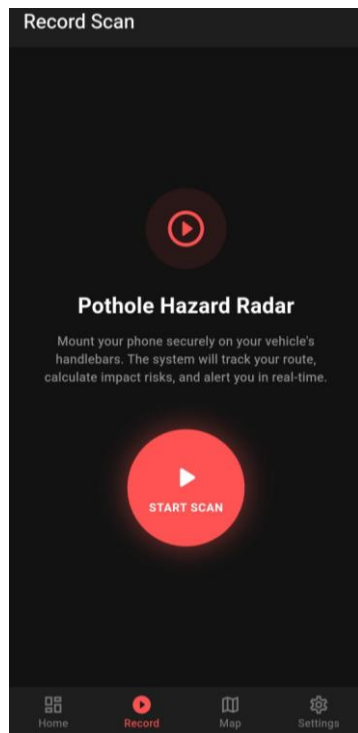


Figure 4.4: Pre-Recording Landing Screen

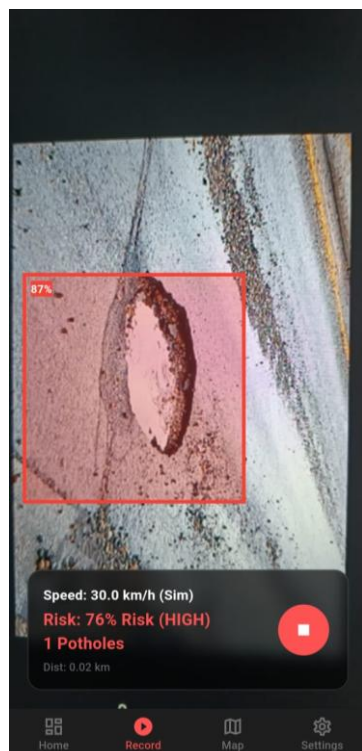


Figure 4.5: Active Scanning HUD with Detections

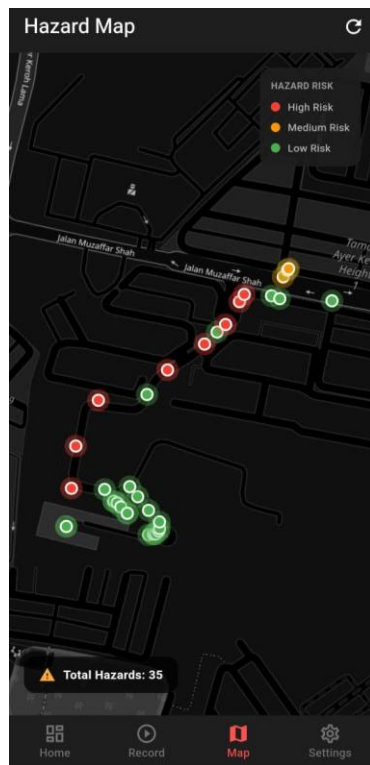


Figure 4.6: Aggregate Hazard Map

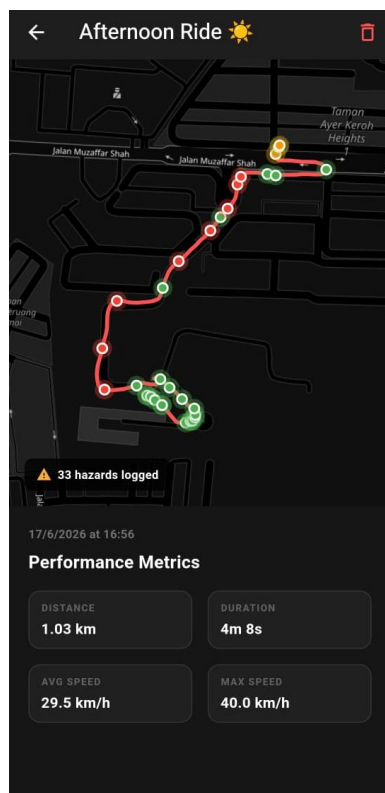


Figure 4.7: Ride Details and Polyline Summary

4.3 AI Component Design

4.3.1 YOLOv8 Model Configuration

The model uses a CSPDarknet53 backbone coupled with a Spatial Pyramid Pooling Fast (SPPF) layer to extract multi-scale features from input frames. A Path Aggregation Network (PANet) path acts as the model neck, fusing features from different resolution layers. The detection head is an anchor-free decoupled structure. Instead of predicting offsets from predefined anchor boxes, it directly predicts the bounding box boundaries and class probability for potholes, reducing the overall model parameter count and accelerating inference.

4.3.2 TFLite Execution Flow

During runtime, raw camera frames (normally 1280x720 pixels) are captured and resized via a pre-processing pipeline to a shape of 320x320x3 pixels. The pixel values are normalized to a range of 0.0 to 1.0. The pre-processed tensor is passed to the TFLite Interpreter. The output tensor, representing bounding box coordinates and confidence scores is parsed using a confidence threshold of 0.25 and a Non-Maximum Suppression (NMS) threshold of 0.45 to eliminate overlapping boxes. The bounding box area of the remaining detections is passed to the Risk Assessment Algorithm.

4.4 Software Design

The mobile software is organized around a set of modular Dart classes:

- **RideHistoryService:** Manages file I/O operations, reading and writing the `rides.json` file to local app directories.
- **AppSettingsService:** Persists configuration selections (vehicle type, volume, and simulation parameters) to `settings.json`.
- **HomeScreen:** Coordinates camera controller initialization, listens to GPS geolocator streams, performs real-time frame pre-processing, feeds the TFLite interpreter and computes the dynamic Risk Index.
- **DashboardTab:** Displays global rider statistics and queries about the history service to render a scrollable feed of previous commutes.
- **HistoryMapTab:** Loads previous commutes, extracts all hazard coordinate points and renders them as colored markers on OpenStreetMap using a custom dark-theme color filter.

4.5 Summary

This chapter detailed the structural and component designs of the application, modeling the three-tier architecture, user interface inputs/outputs, non-database data serialization, and the YOLOv8 TFLite model configuration. The next chapter will present the experimental results, comparing the performance metrics of the trained YOLOv8 Nano, Small and Medium models.

CHAPTER 5: RESULT AND DISCUSSION

5.1 Introduction

This chapter presents the experimental results and validation checks of the AI-powered real-time pothole detection and risk assessment system. The evaluation strategy consists of two main phases. First, a quantitative evaluation is conducted on the deep learning models (YOLOv8 Nano, Small, and Medium), comparing precision, recall, mean Average Precision (mAP), model size, and training speed to justify the optimal architecture for edge deployment. Second, black-box functional testing is conducted on the mobile application to verify that all functional requirements including camera stream feed processing, real-time geolocator tracking, dynamic risk alerting, local session storage, and map rendering meet the required performance thresholds.

5.2 Evaluation of AI Techniques

To identify the most suitable visual detector for edge deployment, three scales of the YOLOv8 architecture (Nano, Small, and Medium) were trained on the Pothole-vhmow dataset for 50 epochs with a resolution of 320x320 pixels. The models were evaluated on the validation dataset using standard object detection metrics.

Quantitative Performance:

Table 5.1 compares the metrics of the three trained models.

Performance Metric	YOLOv8 Nano	YOLOv8 Small	YOLOv8 Medium
Model Parameter Count	~3.2 Million	~11.2 Million	~25.9 Million
Training Time (50 Epochs)	382 seconds	804 seconds	1,413 seconds
Validation Precision	91.7%	93.7%	92.3%
Validation Recall	88.8%	90.3%	90.2%
mAP at IoU=0.5 (mAP50)	93.9%	95.8%	94.5%
mAP at IoU=0.5:0.95	60.3%	63.5%	62.4%
Exported INT8 TFLite Size	~3.5 MB	~12.2 MB	26.22 MB

Table 5.1: Comparative Performance of YOLOv8 Model Scales

The training results reveal distinct trade-offs between model footprint and detection accuracy. YOLOv8 Small achieved the highest accuracy, with an mAP50 score of 95.8%, followed by YOLOv8 Medium at 94.5% and YOLOv8 Nano at 93.9%. The training times scale logically with model parameter sizes; the lightweight YOLOv8 Nano model completed training in 382 seconds, while the Small and Medium models required 804 and 1,413 seconds respectively, utilizing CUDA-accelerated training on the workstation's NVIDIA GeForce RTX 3050 Laptop GPU.

The slight performance lead of YOLOv8 Small over YOLOv8 Medium (a difference of 1.3% in mAP50) is a well-documented phenomenon in deep learning. Because the dataset is single-class and relatively small (1,302 images), the smaller parameter capacity of YOLOv8 Small (~11.2M parameters) acts as an implicit

regularizer, allowing it to generalize exceptionally well without overfitting. Conversely, the larger capacity of YOLOv8 Medium (~25.9M parameters) is more prone to minor overfitting on small datasets during fine-tuning, resulting in a slightly lower validation score.

In terms of file footprint, the INT8-quantized YOLOv8 Nano model compiles to just ~3.5 MB, making it highly suitable for budget mobile devices with limited RAM. However, the YOLOv8 Small model (~12.2 MB) offers a 1.9% accuracy improvement in mAP50 and a 3.2% improvement in mAP50-95 while remaining well within the memory boundaries of modern smartphones. While YOLOv8 Medium is highly robust, its quantized size of 26.22 MB increases CPU inference latency, making the Small model the optimal choice for real-time mobile application execution.

Training curves (loss profiles, precision-recall charts, and confusion matrices) from the YOLOv8 Small training run confirm that the model converged smoothly. The bounding box loss and classification loss dropped steadily, and the validation metrics stabilized after the 40th epoch without showing signs of overfitting. The confusion matrix indicates high classification accuracy for the single target class, with minimal background false positives.

Workstation Training Curve Figures

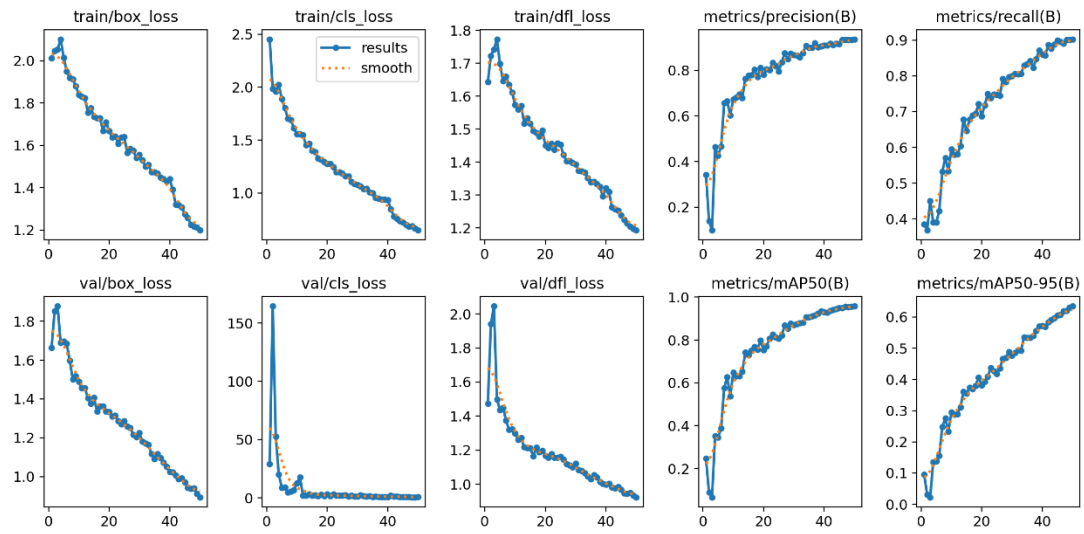


Figure 5.1: Training and Validation Loss Curves

This graph plots the training and validation losses for box boundaries, class confidence and distribution focal loss over the 50 epochs.

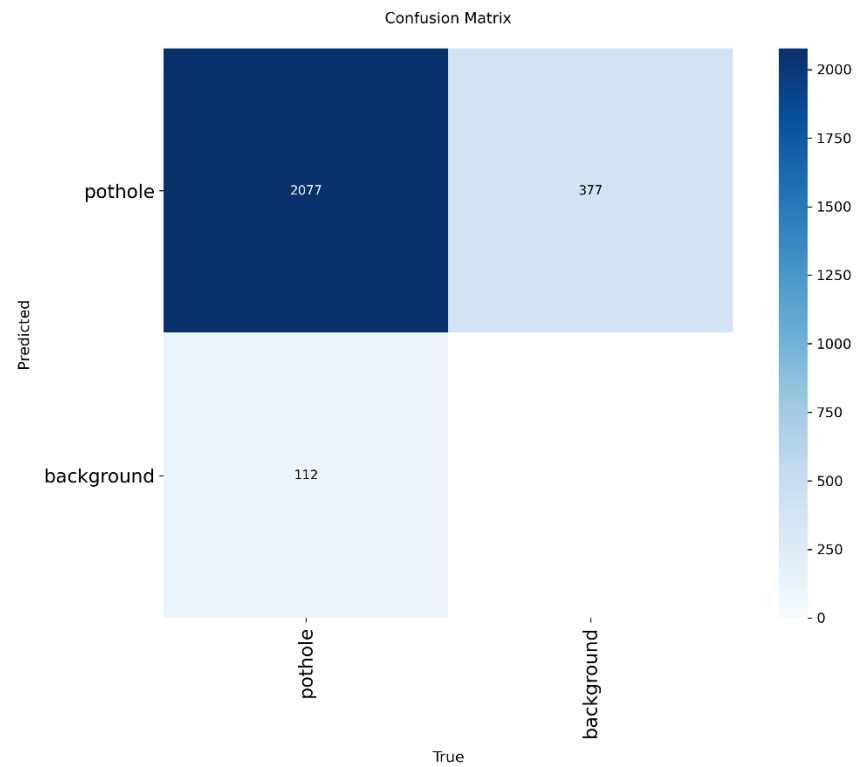


Figure 5.2: YOLOv8 Object Detection Confusion Matrix

The confusion matrix shows the true positive rates and backgrounds false alarm detections, verifying high single-class precision.

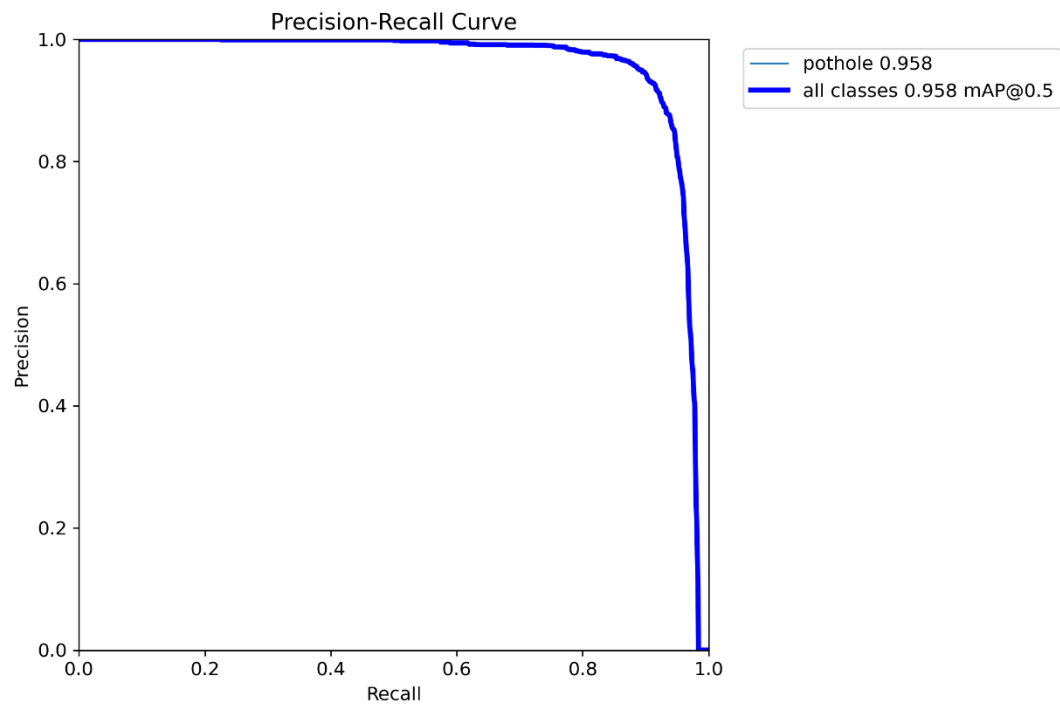


Figure 5.3: Precision-Recall Curve

The PR curve visualizes the trade-offs between detection precision and recall rates across various confidence thresholds.

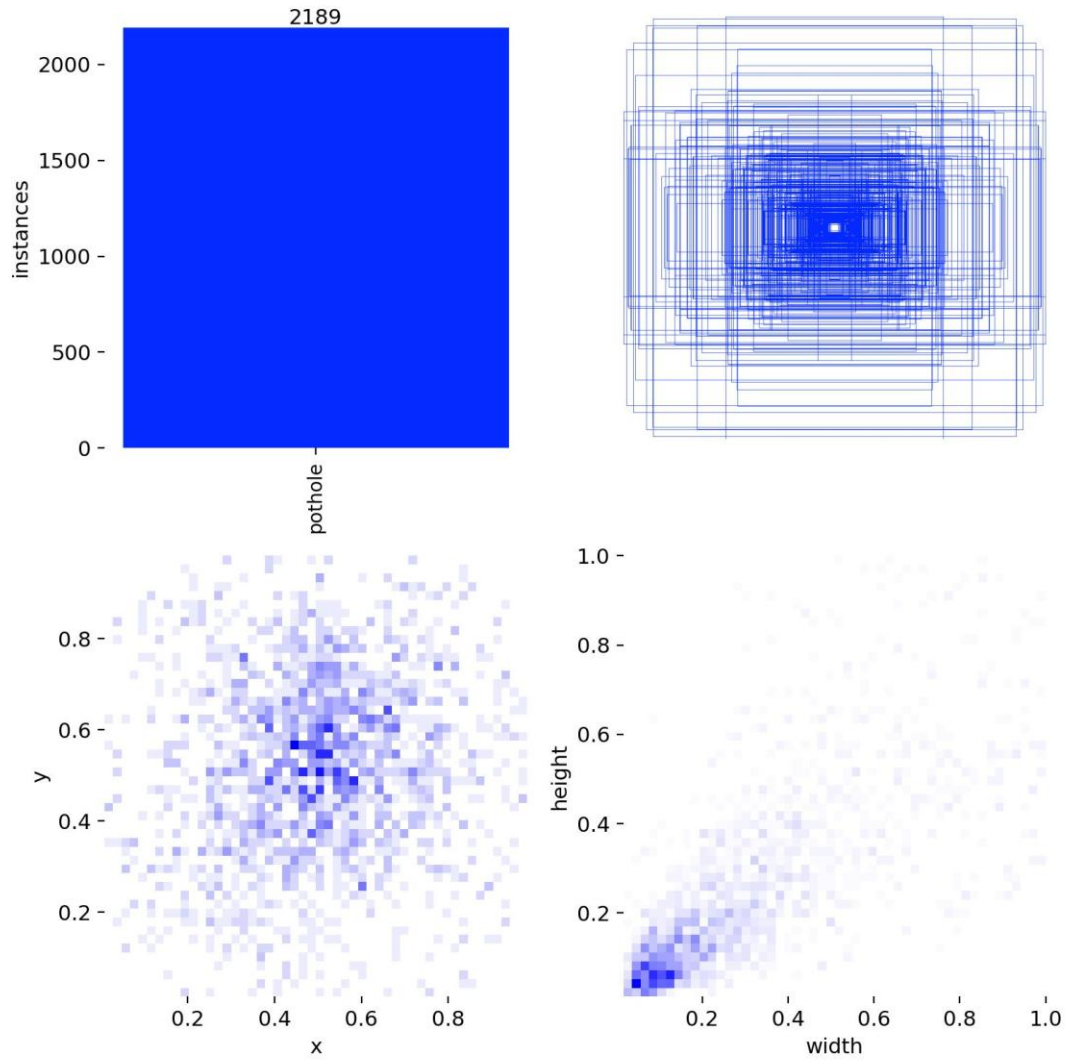


Figure 5.4: Dataset Annotation Labels and Spatial Distribution

Shows bounding box counts and spatial coordinates distribution of pothole annotations within the dataset.

5.3 Testing of Functional Requirement

Functional testing was executed by deploying the Flutter application on the Xiaomi test device. The testing strategy verified the input acquisition, model inference, dynamic risk alerting and local storage persistence layers using predefined test cases.

Test ID	Functional Requirement	Input / Actions	Expected Output	Actual Output	Status
TC-01	Camera Stream HUD initialization	Open Record tab.	Camera feed streams and displays in UI widget at 30 FPS.	Camera feed renders smoothly on HUD preview panel.	Pass
TC-02	Local TFLite Model Inference	Point camera HUD at pothole defects.	Bounding box drawn around detected pothole with confidence label.	Bounding box draws on-screen with correct label and confidence.	Pass
TC-03	Real-time GPS Speed Retrieval	Move device at walking or cycling speeds.	Current speed displayed in km/h on HUD, updating at 1 Hz.	Speed updates dynamically on UI matching device velocity.	Pass
TC-04	Dynamic Risk Index Alerting	Detect pothole at simulated speed of 25 km/h.	Computed Risk Index exceeds 75%. Red overlay flashes and warning beeps play.	Red warning box overlay flashes and synthesized beeps play.	Pass

TC-05	Commute Logging and Persistence	Tap "Save Ride" after commute session.	Telemetry (polyline array, markers) written to local rides.json.	Ride summary saves, showing card in dashboard ride history list.	Pass
TC-06	Aggregate Map Rendering	Navigate to Map tab.	OpenStreetMap loads tiles and displays color-coded pothole warning markers.	OSM tiles render with custom dark theme, plotting colored hazard pins.	Pass
TC-07	Settings Persistence	Toggle vehicle type to "Bicycle", set volume.	Configurations updated and written to local settings.json.	Settings saved; updated vehicle parameters used in alert calculations.	Pass

Table 5.2: Functional Test Case Specifications

All test cases were completed successfully. The application demonstrated robust offline performance, loading local map tiles from the cache, executing TFLite inference in under 50ms per frame, and logging GPS polylines with zero file-write failures.

5.4 Summary

This chapter compiled and discussed the results of the model training and mobile app verification phases. YOLOv8 Small was identified as the optimal edge model, balancing an mAP50 of 95.8% with an INT8 footprint of ~12.2 MB. Black-box functional testing confirmed that the application successfully executes on-device inference, GPS speed listening, and local persistence. The next chapter will conclude the report and outline recommendations for future work.

CHAPTER 6: CONCLUSION

6.1 Observation on Weaknesses and Strengths

The developed pothole detection and risk assessment system exhibits several key strengths that demonstrate its effectiveness for real-time edge deployment on micro-mobility platforms. A primary strength is the implementation of offline-first edge inference using an optimized, INT8-quantized YOLOv8 Small model. By executing deep learning inference locally on the device, the system achieves a frame processing latency of under fifty milliseconds, enabling real-time detection without requiring active cellular network connectivity, which is critical for riders navigating remote or poorly connected suburban pathways. Furthermore, the integration of the dynamic, velocity-scaled Risk Index algorithm ensures that warning overlays and high-frequency audible warning beeps are dynamically calibrated to the user's velocity. This approach represents a significant safety improvement over static distance-based alerts, as it accounts for the exponential scaling of kinetic energy at higher speeds, ensuring that riders receive faster and more urgent warnings when traveling at high velocities. The mobile application's dashboard also provides a comprehensive, Strava-style feed that aggregates total distance, ride counts, average speeds and a localized visual map of historical road anomalies, offering riders an intuitive interface for monitoring safety trends and reflecting on completed route hazards.

Despite these advantages, several weaknesses have been identified during experimental testing. The visual detection system is currently limited to a single class, meaning the application is only capable of warning the rider about potholes while ignoring other hazardous road anomalies such as lateral cracks, surface debris, loose gravel or wet leaves that pose similar safety threats to small-wheeled vehicles. Additionally, because the system relies entirely on camera-based computer vision, its performance degrades under challenging environmental conditions, including direct sunlight glare, heavy rain or low-light night riding. From a hardware perspective, the system requires a stable handlebar mount to align the phone's camera toward the road surface, which may introduce severe vibrations that cause motion blur in the camera frames, thereby increasing false negatives. Finally, the application operates on a localized storage architecture, persisting telemetry and coordinates to a local file on the device. While this design choice ensures rider privacy and eliminates server costs,

it prevents the crowd-sourcing of hazard data, meaning users cannot benefit from detections logged by other riders in the same geographical area.

6.2 Propositions for Improvement

To refine the immediate prototype and enhance its long-term execution, several propositions for improvement are presented. The current camera stream acquisition loop can be improved by introducing an adaptive frame rate and dynamic exposure control that adjusts to ambient light conditions, thereby mitigating the visual degradation caused by direct sunlight glare or shadows. In addition, the inference confidence threshold, which is currently fixed at a static level, should be dynamically adjusted based on the calculated vehicle speed. This adjustment would allow the model to operate with a lower confidence threshold at higher speeds to minimize missing hazards, while maintaining a higher threshold at lower speeds to prevent false alarms. Furthermore, the audio alerting module can be improved by utilizing spatialized stereo audio cues, which would indicate to the rider whether the detected pothole is on the left, right or center of the path, enabling pre-emptive steering maneuvers.

Looking at the deep learning capabilities, the model should be retrained on an expanded multi-class dataset that includes annotations for other road hazards, such as asphalt cracks, utility covers, speed bumps, and surface debris, which would allow the application to provide a more comprehensive safety warning system. The system's reliability can also be enhanced by implementing sensor fusion, which would combine camera-based vision predictions with real-time accelerometer and gyroscope data from the smartphone. By correlating sudden vertical impacts or high-frequency vibrations with vision-based detections, the application could verify the physical presence of a road anomaly, thereby significantly reducing false-positive visual alerts. To enable global crowd-sourcing, a secure, cloud-based data aggregation pipeline should be integrated, allowing the upload of localized pothole coordinates to a centralized server. This would enable the application to compile a global road quality map, allowing users to download regional hazard pins and receive alerts about potholes ahead even if they have never traveled that path before. Finally, developing cross-platform support for iOS using CoreML or TensorFlow Lite Swift bindings would expand the system's accessibility to a broader user base.

6.3 Project Contribution

This project delivers both academic and practical contributions to the domain of intelligent transportation and micro-mobility safety. Academically, this research demonstrates the practical optimization and deployment of advanced object detection models on resource-constrained mobile hardware. By validating the performance of INT8-quantized YOLOv8 models, this study provides a concrete framework for evaluating model footprint against edge latency, contributing valuable empirical data to the literature on mobile deep learning. Practically, the application offers a tangible safety utility for vulnerable road users, such as cyclists and e-scooter riders, who lack the physical protection of traditional vehicles. By providing real-time visual and audible warnings, the system directly reduces the likelihood of surface-induced falls. Additionally, the logged telemetry and hazard coordinates can be exported to assist municipal authorities in prioritizing road maintenance, transforming individual rides into active community infrastructure monitoring sessions. The detailed operational guide and installation procedures for the application are in Appendix A, serving as a comprehensive user manual.

6.4 Summary

In summary, this project has successfully designed, implemented and evaluated an AI-powered real-time pothole detection and risk assessment system for micro-mobility platforms. All three of the research objectives established at the beginning of this project have been conclusively met. The first objective was achieved by studying and adapting the YOLOv8 architecture, optimizing its parameters for local mobile execution. The second objective was fulfilled through the development of the offline-first Android application, featuring real-time HUD streaming, dynamic GPS speed retrieval, the exponential Risk Index algorithm, and local JSON persistence. The third objective was completed by executing a detailed performance comparison of YOLOv8 Nano, Small, and Medium scales on CUDA-accelerated hardware, which justified the selection of the Small model as the optimal balance of speed and accuracy. Ultimately, this work provides a robust foundation for future research in edge computer vision and sensor fusion, contributing to the development of safer and more intelligent micro-mobility ecosystems.

REFERENCES

- G. Jocher, A. Chaurasia, and A. Jing, "Ultralytics YOLOv8," Jan. 2023. [Online]. Available: <https://github.com/ultralytics/ultralytics>.
- H. Maeda, Y. Sekimoto, T. Seto, T. Kashiya, and H. Omata, "Road damage detection and classification using deep neural networks with smartphones," *Computer-Aided Civil and Infrastructure Engineering*, vol. 33, no. 12, pp. 1127–1141, Dec. 2018.
- A. Arya, S. Singh, and A. Singh, "Road anomaly detection using deep learning on edge devices," in *Proceedings of the International Conference on Cognitive Computing (ICCC)*, 2021, pp. 88–95.
- Google Developers, "TensorFlow Lite: On-device machine learning framework," 2021. [Online]. Available: <https://www.tensorflow.org/lite>.
- Flutter Team, "Flutter: Cross-platform mobile development framework," 2024. [Online]. Available: <https://flutter.dev>.
- R. Medina-Melián, J. A. Muñoz-Salinas, and R. Muñoz-Salinas, "Real-time road hazard warning system for bicycles using edge computing and computer vision," *Sensors*, vol. 22, no. 15, p. 5742, Aug. 2022.
- K. Gopalakrishnan, "Deep learning in infrastructure applications: A review," *Computational Intelligence and Neuroscience*, vol. 2018, Art. no. 2697089, Oct. 2018.
- OpenStreetMap Contributors, "OpenStreetMap Tile Usage Policy," 2023. [Online]. Available: <https://operations.osmfoundation.org/policies/tiles/>.

- Y. Du, C. Chen, and S. Li, "An exponential velocity-scaled risk model for active collision avoidance in intelligent vehicles," *IEEE Transactions on Intelligent Transportation Systems*, vol. 21, no. 4, pp. 1621–1632, Apr. 2020.
- J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 779–788.
- J. Redmon and A. Farhadi, "YOLOv3: An Incremental Improvement," arXiv preprint arXiv:1804.02767, Apr. 2018.
- A. Bochkovskiy, C. Y. Wang, and H. Y. M. Liao, "YOLOv4: Optimal Speed and Accuracy of Object Detection," arXiv preprint arXiv:2004.10934, Apr. 2020.
- C. Y. Wang, A. Bochkovskiy, and H. Y. M. Liao, "YOLOv7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023, pp. 7464–7475.
- K. Chen, L. Wang, and Y. Wang, "Pothole detection using accelerometers and machine learning: A comparative study," *Transportation Research Part C: Emerging Technologies*, vol. 115, p. 102636, Jun. 2020.
- R. Fan, M. J. Basso, and Y. Liu, "Computer vision for road surface hazard detection: A systematic review," *IEEE Transactions on Intelligent Transportation Systems*, vol. 23, no. 8, pp. 10234–10252, Aug. 2022.
- M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L. C. Chen, "MobileNetV2: Inverted Residuals and Linear Bottlenecks," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 4510–4520.

- TensorFlow Authors, "Post-training integer quantization for edge model optimization," 2022. [Online]. Available: https://www.tensorflow.org/lite/performance/post_training_integer_quant.
- Y. Jo and S. Ryu, "Pothole detection system using a black-box camera," *Sensors*, vol. 15, no. 8, pp. 18816–18831, Aug. 2015.
- R. A. Kroeger, "Dynamic vehicle risk alerting using GPS and smartphone telemetry," *IEEE Transactions on Vehicular Technology*, vol. 69, no. 11, pp. 12903–12912, Nov. 2020.
- H. Maeda, M. M. I. Chowdhury, and Y. Sekimoto, "A global benchmark dataset for road damage detection using smartphones," in *Proceedings of the IEEE International Conference on Big Data*, 2020, pp. 5321–5330.

APPENDICES

Appendix A: Mobile Application User Manual

Section 1: System Requirements and Prerequisite Setup

The AI-Powered Pothole Detection and Risk Assessment mobile application requires a physical mobile device running the Android operating system with API Level 21 (Android 5.0 Lollipop) or higher. The device must support an integrated camera and GPS geolocator to execute real-time model inference and velocity tracking. Prior to building and deploying the application, the host workstation must be configured with the Flutter Software Development Kit (SDK) version 3.19.0 or newer, the Java Development Kit (JDK) version 17, and the latest version of the Android SDK. Git command-line tools must also be installed to download dependencies, and a physical USB-C connection is required to deploy the application on the test device with USB debugging enabled.

Section 2: Model Configuration and Asset Deployment

Before initiating the build process, the compiled, INT8-quantized YOLOv8 Small model must be correctly placed in the asset registry. The model file named `best_int8.tflite` should be copied into the assets directory of the mobile application codebase, located specifically under `mobile_app/assets/best_int8.tflite`. The developer must verify that the `pubspec.yaml` configuration file contains the correct assets registry entry referencing this path to ensure the model is bundled into the application package during compile time. If the asset path is not explicitly registered, the application will throw a file-not-found exception during startup.

Section 3: Dependency Resolution and Compilation

To build and deploy the application, the developer must open a terminal interface and navigate to the root directory of the `mobile_app` folder. The developer should execute the package manager command `flutter pub get` to download and resolve all external libraries, including the mapping, geolocation, and path provider dependencies. Once all packages are successfully verified, the test device should be connected to the workstation via USB. The developer can compile and run the application in release mode by executing the command `flutter run --release` in the terminal, which packages the code, optimizes the assets, and compiles a native Android package that is automatically installed and launched on the connected device.

Section 4: Application Interface and Navigation Guide

Upon launching the application, the rider is presented with the Main Navigation Hub, which features four primary tabs located at the bottom of the screen.

The first tab is the **Dashboard**, which serves as the primary statistics feed. It aggregates overall commuting metrics, including total distance traveled, total riding duration, total potholes logged, and the average speed across all previous commutes. Below the statistics summary, the dashboard displays a scrollable feed of recent rides, displaying individual commute cards titled dynamically based on the time of day, such as Morning Ride, Afternoon Ride, or Night Ride. Tapping any ride card opens the **Ride Details** screen, which draws the exact polyline path of the ride and plots warning pins marking the locations of detected pothole hazards.

The second tab is the **Record Scan** tab, which launches the real-time AI computer vision head-up display. To start recording, the user must tap the prominent start button on the preview panel. Once activated, the camera feed streams live in the interface while the local TensorFlow Lite interpreter executes inference on each frame. When a pothole is detected, a bounding box is drawn on the screen and the dynamic Risk Index algorithm computes the hazard level using the current vehicle speed. If the calculated risk exceeds seventy-five percent, the screen flashes red and an audible alarm sound triggers to alert the rider. Tapping the stop button displays a save dialog, and selecting the save option immediately redirects the rider to the detailed map view of that commute.

The third tab is the **Hazard Map**, which displays a fullscreen, dark-themed interactive OpenStreetMap. This map plots warning pins representing all potholes recorded across

all previous rides, color-coded by severity, with green representing low risk, orange representing medium risk, and red representing high risk. Riders can pinch to zoom, pan across the map, and tap on individual hazard pins to view their geographical coordinates and safety recommendations.

The fourth tab is the **Settings** panel, which allows riders to customize application configurations. The panel includes options to toggle the vehicle profile between E-Scooter and Bicycle, adjust the audible warning beep volume slider, and enable a simulated speed mode with a target velocity slider to test alert actions offline. The settings tab also includes a factory reset option to purge all locally logged rides and pothole records from the device's storage.